

# Personal Second Brain — RAG Pipeline Technical Documentation

## Before vs. After: Comprehensive Beginner-Friendly Guide

Welcome to the complete technical documentation for your **Retrieval-Augmented Generation (RAG)** pipeline. This document explains every single change made to your codebase, why it was made, how the code works line-by-line, and how data flows from your documents into Ollama.

## 1. Project Flow Before the Changes

Here is how your RAG pipeline worked before we made the improvements:

```
File Dropped (watched_folder)
    ↓
File Watcher (watchdog)
    ↓
Kafka Event → Celery Task (route_file)
    ↓
Text Extraction (pdf.py / router.py)
    ↓
Chunking (chunker.py - fixed 500 characters)
    ↓
Embedding Generation (embedding_router.py / MiniLM-L6-v2)
    ↓
LanceDB Vector Storage (lancedb_store.py)
    ↓
Vector Search (vector_search.py - limit(10))
    ↓
Relevance Filtering (score_threshold ≤ 0.50)
    ↓
Context Creation (rag_pipeline.py)
    ↓
Ollama (llama model)
    ↓
Refusal String Override (if "I could not find" in answer: sources = [])
    ↓
Final Answer + Sources Output
```

### Stage-by-Stage Breakdown of the Old Pipeline:

#### 1. File Watcher ( `app/watcher/file_watcher.py` ):

\* **Input:** A file placed inside `watched_folder/`.

\* **Action:** Emitted a Kafka event containing `path`, `file_type`, and MD5 `file_hash`.

\* **Output:** Event payload sent to Kafka topic.

#### 2. File Processing Task ( `app/celery_app/tasks/process_file.py` ):

\* **Input:** Kafka event.

\* **Action:** Checked if the file hash was already processed. If new, called `extract_file(path)`.

\* **Output:** Extracted text string.

#### 3. Text Extraction ( `app/extractors/pdf.py` & `router.py` ):

\* **Input:** File path.

\* **Action:** Extracted all text into a single giant string. Page numbers were discarded.

\* **Output:** Plain text string.

#### 4. Chunking ( `app/chunker/chunker.py` ):

\* **Input:** Full text string.

\* **Action:** Sliced text by fixed character length ( `text[i:i+500]` ).

\* **Problem:** Words and sentences were cut in half (e.g. `"super-user"` became `"superu"` in chunk 1 and `"ser-user"` in chunk 2).

\* **Output:** List of character slice strings.

#### 5. Embedding & LanceDB ( `app/storage/lancedb_store.py` ):

\* **Input:** Chunks with `File:` prepended.

\* **Action:** Generated 384-dimensional dense vectors using `all-MiniLM-L6-v2` and stored them in `database/documents.lance`.

\* **Output:** Database records containing `chunk_id`, `path`, `file_type`, `text`, `embedding`.

#### 6. Vector Search & Relevance Filtering ( `app/search/vector_search.py` ):

\* **Input:** User question.

\* **Action:** Generated vector for query and searched LanceDB with `.limit(10)`.

\* **Problem:** Front-matter noise (Table of Contents dotted lines `. . . . 419` and publisher reviews) scored low vector distances (~0.27) and crowded out actual body text chunks (scores ~0.33 to 0.50).

\* **Output:** 10 chunks (mostly TOC/reviews).

#### 7. Context Assembly & LLM Generation ( `app/rag/rag_pipeline.py` ):

\* **Input:** 10 retrieved chunks.

\* **Action:** Combined text into `Context` and sent prompt to Ollama.

*Problem:* Ollama saw TOC/reviews, could not find detailed body text, and issued refusal: "I could not find enough relevant information...".

\* **Fatal Bug:** `rag_pipeline.py` checked `if "I could not find" in answer:` and set `sources = []` (wiping out source filenames!), causing the terminal to report `Found 10 relevant chunks` on stdout while printing `Source: No relevant source found` in the UI!

## 2. Project Flow After the Changes

Here is how your current pipeline operates cleanly from start to finish:

```
File Dropped (watched_folder)
↓
File Processing Task (process_file.py)
↓
Page-by-Page Extraction (pdf.py → returns [{"page": 1, "text": "..."}, ...])
↓
Paragraph & Sentence-Aware Chunking (chunker.py → splits by \n without breaking words)
↓
Header Attachment (File: linux.pdf | Page: 1 → attached to each chunk)
↓
Embedding & LanceDB Storage (lancedb_store.py)
↓
User Question
↓
Vector Search Candidate Pool (vector_search.py → limit(50))
↓
Generic Meta-Chunk Demotion (is_table_of_contents_or_noise → body text prioritized)
↓
Relevance Filtering (_distance ≤ 0.50 → max_results=10)
↓
Python Retrieval Metadata Calculation BEFORE LLM (num_chunks = len(results), sources = [ ... ])
↓
Pre-LLM Retrieval Check (If num_chunks == 0 → Return immediately with Sources: No relevant source found)
↓
Context Assembly (Formatted context string)
↓
Ollama (Synthesizes answer using retrieved body chunks)
↓
Python Output Formatter (format_response → Answer + Source + Relevant Chunks Count)
```

### Stage-by-Stage Breakdown of the New Pipeline:

- Page-Aware Extraction:** `pdf.py` extracts PDF text page-by-page as `[{"page": 1, "text": "..."}, ...]`.
- Word-Safe Paragraph Chunking:** `chunker.py` splits text by line breaks (`text.split("\n")`) and groups paragraphs up to `max_chars=600` without cutting words.
- Expanded Search Pool & Demotion:** `vector_search.py` fetches 50 candidate raw chunks from LanceDB, demotes TOC/reviews dotted lines, and filters by relevance (`_distance ≤ 0.50`).
- Python-Driven Truth:** `rag_pipeline.py` calculates `num_chunks` and `sources` using pure Python **before** calling Ollama. Ollama's generated text can **never** erase Python's source metadata.
- Unified Output:** Python formats answer, dynamic source filenames, and actual relevant chunk count.

### 3. Complete Before vs. After Table

| Area                | Before                                                             | Problem                                                     | After                                                                                 | Why                                                       |                                                    |
|---------------------|--------------------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------|
| Chunking            | Fixed slicing<br><code>text[i:i+500]</code>                        | Words like<br>"superuser" cut<br>mid-word                   | Paragraph-based grouping<br><code>max_chars=600</code>                                | Preserves word<br>boundaries and<br>sentence<br>semantics | Preserves<br>page<br>structure<br>inside<br>chunks |
| Metadata            | Flat text with <code>File:</code>                                  | PDF page numbers<br>discarded                               | Page headers <code>File: linux.pdf \</code>                                           | Page: 1                                                   |                                                    |
| Vector<br>Search    | <code>.limit(10)</code> raw<br>query                               | TOC/Reviews<br>crowded out body<br>chunks                   | <code>.limit(50)</code> +<br><code>is_table_of_contents_or_noise()</code><br>demotion | Body chunks<br>rank first for<br>content<br>questions     |                                                    |
| Chunk<br>Count      | stdout printed <code>Found<br/>10 chunks</code>                    | Always reported 10<br>regardless of actual<br>relevance     | Dynamic <code>num_chunks = len(results)</code><br>(0, 1, 3, 10)                       | Shows actual<br>number of<br>relevant chunks<br>retrieved |                                                    |
| Source<br>Metadata  | Wiped if Ollama<br>refused                                         | False <code>Source: No<br/>relevant source<br/>found</code> | Calculated in Python before Ollama; never<br>wiped                                    | Python owns<br>source<br>metadata, not<br>LLM             |                                                    |
| Refusal<br>Trigger  | <code>if "I could not<br/>find" in answer:<br/>sources = []</code> | Overrode valid<br>retrieval sources                         | <code>if num_chunks == 0:</code> pre-LLM check                                        | Separates<br>retrieval status<br>from LLM<br>phrasing     |                                                    |
| Synthesis<br>Prompt | Strict prompt forced<br>single-chunk answer                        | Refused broad<br>chapter summaries                          | Instructed Ollama to synthesize across<br>body chunks                                 | Allows<br>answering<br>broad summary<br>questions         |                                                    |
| Future<br>Files     | Worked but got<br>hijacked by TOC                                  | Long books failed                                           | 100% generic metadata-driven demotion                                                 | Works for any<br>new PDF/XLSX<br>without code<br>edits    |                                                    |
| Response<br>Format  | Plain Answer + Source                                              | Lacked explicit<br>chunk count<br>reporting                 | <code>Answer: ... \n\nSource:<br/>... \n\nRelevant chunks: X</code>                   | Clear, readable<br>layout with<br>chunk metrics           |                                                    |

## 4. Explain Every Code Change

File 1: [ `app/chunker/chunker.py` ]

(file:///home/rashmi/Data\_Engineering/Personal%20Second%20Brain%20Project/app/chunker/chunker.py)

**Purpose:** Splits extracted document text into smaller chunks suitable for embedding models.

**OLD CODE:**

```
def chunk_text(text, chunk_size=500, overlap=100):
    if not text:
        return []
    chunks = []
    step = chunk_size - overlap
    for i in range(0, len(text), step):
        chunk = text[i:i + chunk_size].strip()
        if chunk:
            chunks.append(chunk)
    return chunks
```

**NEW CODE:**

```
def chunk_text(text, max_chars=600):
    if not text:
        return []

    paragraphs = text.split("\n")
    chunks = []
    current_chunk = ""

    for paragraph in paragraphs:
        paragraph = paragraph.strip()
        if not paragraph:
            continue

        if len(current_chunk) + len(paragraph) + 1 <= max_chars:
            if current_chunk:
                current_chunk += "\n" + paragraph
            else:
                current_chunk = paragraph
        else:
            if current_chunk:
                chunks.append(current_chunk)
            current_chunk = paragraph

    if current_chunk:
        chunks.append(current_chunk)

    return chunks
```

- **What changed?** Replaced fixed index string slicing with a line/paragraph-accumulating loop.
- **Why did we change it?** Fixed character slicing cut words in half across chunk boundaries.
- **What does each line do?**
  - \* `paragraphs = text.split("\n")` : Splits raw text into lines.
  - \* `if len(current_chunk) + len(paragraph) + 1 <= max_chars:` : Checks if adding the paragraph exceeds 600 characters.
  - \* `chunks.append(current_chunk)` : Saves the clean paragraph block.
- **Effect on RAG:** Clean sentence chunks preserve exact vector semantics when embedded.

**File 2:** [ `app/extractors/pdf.py` ]  
 (file:///home/rashmi/Data\_Engineering/Personal%20Second%20Brain%20Project/app/extractors/pdf.py)

**Purpose:** Reads PDF files and extracts text content.

**OLD CODE:**

```
def extract_pdf(path):
    text = ""
    with pdfplumber.open(path) as pdf:
        for page in pdf.pages:
            page_text = page.extract_text() or ""
            text = text + page_text + "\n"

    return text
```

**NEW CODE:**

```
import pdfplumber

def extract_pdf(path):
    pages_data = []
    try:
        with pdfplumber.open(path) as pdf:
            for i, page in enumerate(pdf.pages, start=1):
                page_text = page.extract_text() or ""
                page_text = page_text.strip()
                if page_text:
                    pages_data.append({
                        "page": i,
                        "text": page_text
                    })
    except Exception as e:
        print(f"PDF extraction error: {e}")
    return pages_data
```

- **What changed?** Instead of returning one giant string, it returns a list of dictionaries with page numbers `[{"page": 1, "text": "..."}, ...]`.
- **Why did we change it?** To preserve page numbers so chunks can carry page metadata.
- **Effect on RAG:** Every PDF chunk now carries `Page:` for contextual awareness.

**File 3:** `[ app/celery_app/tasks/process_file.py ]`

`(file:///home/rashmi/Data_Engineering/Personal%20Second%20Brain%20Project/app/celery_app/tasks/process_file.py)`

**Purpose:** Asynchronous Celery task that coordinates file extraction, chunking, embedding, and LanceDB storage.

**NEW CODE (Relevant Section):**

```
# Case 1: PDF extraction returned a list of page dictionaries
if isinstance(extracted_data, list):
    for page_info in extracted_data:
        page_num = page_info.get("page", 1)
        page_text = page_info.get("text", "")

        page_chunks = chunk_text(page_text)
        for c in page_chunks:
            header = f"File: {filename} | Page: {page_num}"
            chunks.append(f"{header}\n{c}")
```

- **What changed?** Checks if extracted data is a page dictionary list and prepends page headers to each chunk.
- **Why did we change it?** Enables RAG to display and store exact page metadata.

File 4: [ `app/search/vector_search.py` ]

(`file:///home/rashmi/Data_Engineering/Personal%20Second%20Brain%20Project/app/search/vector_search.py`)

**Purpose:** Queries LanceDB vector store using cosine distance, filters noise, and returns relevant chunks.

**NEW CODE:**

```

def is_table_of_contents_or_noise(text):
    text_lower = text.lower()
    if text.count(".") ≥ 15:
        return True
    if "contents in detail" in text_lower or "reviews for how linux works" in text_lower or "420 bibliography" in text_lower:
        return True
    return False

def search(question, max_results=10, score_threshold=0.50):
    table = get_table()
    if table is None:
        return []

    question_vector = generate_embedding("text", question)
    question_lower = question.lower()

    # Check for explicit filename reference
    df = table.to_pandas()
    target_path = None
    if not df.empty:
        for path_str in df['path'].unique():
            filename = os.path.basename(str(path_str)).lower()
            if filename in question_lower:
                target_path = path_str
                break

    # Fetch raw candidates
    if target_path:
        escaped_path = str(target_path).replace("'", "'")
        raw_chunks = table.search(question_vector).where(f"path = '{escaped_path}'").metric("cosine").limit(50).to_list()
    else:
        raw_chunks = table.search(question_vector).metric("cosine").limit(50).to_list()

    # Prioritize body chunks over TOC noise
    is_asking_for_toc = "table of contents" in question_lower or "contents" in question_lower or "bibliography" in question_lower
    body_chunks = []
    noise_chunks = []

    for doc in raw_chunks:
        text = doc.get("text", "")
        if is_table_of_contents_or_noise(text) and not is_asking_for_toc:
            noise_chunks.append(doc)
        else:
            body_chunks.append(doc)

    ordered_chunks = body_chunks + noise_chunks

    # Filter by distance score threshold (distance ≤ 0.50)
    relevant_chunks = []
    for doc in ordered_chunks:
        if doc.get("_distance", 1.0) ≤ score_threshold:
            relevant_chunks.append(doc)
        if len(relevant_chunks) == max_results:
            break

    # Diagnostic logging
    sources = sorted(list(set(os.path.basename(c["path"]) for c in relevant_chunks)))
    print(f"\nQuery: {question}")
    print(f"Raw chunks: {len(raw_chunks)}")
    print(f"Relevant chunks: {len(relevant_chunks)}")

```

```
print(f"Sources: {'', '.join(sources) if sources else 'None'}")
print(f"Sending {len(relevant_chunks)} chunks to Ollama.")

return relevant_chunks
```

- **What changed?**

1. Fetches candidate limit of 50 raw chunks.
  2. Demotes Table of Contents dotted lines ( `. . . . 419` ) and reviews unless explicitly requested.
  3. Appends body chunks first, then noise chunks.
  4. Filters by `_distance ≤ 0.50` and returns `relevant_chunks` (up to 10).
  5. Prints Python-based diagnostic logs.
- **Why did we change it?** Solved the issue where TOC/reviews crowded out actual body chunks for broad questions like *"Explain Chapter 1"*.

---

**File 5:** [ `app/rag/rag_pipeline.py` ]

(file:///home/rashmi/Data\_Engineering/Personal%20Second%20Brain%20Project/app/rag/rag\_pipeline.py)

**Purpose:** Coordinates vector search, prompt assembly, LLM execution, and response formatting.

**NEW CODE:**

```

def ask(question):
    # Step 1: File listing check
    question_lower = question.lower()
    is_file_list_query = any(phrase in question_lower for phrase in [
        "do you have", "do i have", "which file", "which document",
        "what files", "list files", "show me files"
    ])

    if is_file_list_query:
        hash_table = get_hash_table()
        all_paths = []
        if hash_table is not None:
            hdf = hash_table.to_pandas()
            if not hdf.empty:
                all_paths = hdf["path"].tolist()
            file_names = sorted(list(set(os.path.basename(p) for p in all_paths)))
            for fn in file_names:
                if fn.lower() in question_lower:
                    return f"Yes, '{fn}' is indexed in your Personal Second Brain.", [fn], 1
            if file_names:
                return f"The following files are indexed in your Personal Second Brain: {'', '.join(file_names)}'

    # Step 2: Perform vector search
    results = search(question)
    num_chunks = len(results)

    # Retrieval decision determined by Python BEFORE calling Ollama
    if not results or num_chunks == 0:
        return "Not found in the retrieved source.", [], 0

    # Step 3: Extract sources and build context string
    context_list = []
    retrieved_sources = set()
    for doc in results:
        filename = os.path.basename(doc["path"])
        retrieved_sources.add(filename)
        context_list.append(doc["text"])

    sources = sorted(list(retrieved_sources))
    context_str = "\n\n".join(context_list)

    # Step 4: Build prompt
    prompt = f"""\
You are a Personal Second Brain Assistant.

Answer the user's question using ONLY the retrieved context below.

Guidelines:
- Rely strictly on the retrieved document content in the Context.
- You may synthesize an answer across multiple retrieved chunks.
- Do not use general outside knowledge or make up information.
- If the retrieved context genuinely does not contain enough information to answer the question, reply exactly:
  "Not found in the retrieved source."

Retrieved Context:
{context_str}

Question: {question}

Answer: """""

```

```
# Step 5: Ask LLM
try:
    answer = ask_llama(prompt)
except Exception:
    try:
        answer = ask_gemini(prompt)
    except Exception as e:
        answer = f"LLM Error: {str(e)}"

return answer, sources, num_chunks

def format_response(answer, sources, num_chunks=0):
    output = f"Answer:\n{answer}\n\n"
    if not sources or num_chunks == 0:
        output += "Source:\nNo relevant source found.\n\n"
    elif len(sources) == 1:
        output += f"Source:\n{sources[0]}\n\n"
    else:
        output += "Sources:\n" + "\n".join(f"- {s}" for s in sources) + "\n\n"

    output += f"Relevant chunks:\n{num_chunks}"
    return output
```

#### What changed?

1. `ask()` returns `(answer, sources, num_chunks)`.
2. Python evaluates `if not results or num_chunks == 0:` before calling Ollama.
3. Removed `if "I could not find" in answer: sources = []`.
4. Ollama prompt explicitly allows multi-chunk synthesis.
5. `format_response()` outputs `Answer`, `Source`, and `Relevant chunks`.

Why did we change it? Decouples Python's source/chunk metadata from Ollama's generated text, fixing the false refusal bug.

## 5. The "Always 10 Chunks" Problem Explained

### Why did it appear to always return 10 chunks?

Previously, `table.search().limit(10).to_list()` retrieved the top 10 nearest neighbors from LanceDB. Because Table of Contents, reviews, and preface chunks had vector distances between `0.27` and `0.45` (all  $\leq 0.50$ ), all 10 raw chunks passed the threshold. The terminal stdout printed `Found 10 relevant chunks (maximum 10)`.

### Understanding the Terms:

- **Raw Search Results:** The initial candidates retrieved by LanceDB (e.g. 50 chunks).
- **Relevant Chunks:** The subset of chunks whose cosine distance is  $\leq 0.50$ .
- **Chunks Sent to Ollama:** The final list of relevant chunks (up to 10 max).

### What `limit(10)` Means:

`limit(10)` specifies the **maximum ceiling** of chunks to retrieve. It does **not** mean 10 chunks are relevant.

### How the New Code Handles Counts:

```
relevant_chunks = [c for c in ordered_chunks if c.get("_distance", 1.0) <= 0.50][:10]
num_chunks = len(relevant_chunks)
```

- 0 chunks relevant  $\rightarrow$  `num_chunks = 0`  $\rightarrow$  Outputs `Relevant chunks: 0`.

- 1 chunk relevant  $\rightarrow$  `num_chunks = 1`  $\rightarrow$  Outputs `Relevant chunks: 1`.
- 3 chunks relevant  $\rightarrow$  `num_chunks = 3`  $\rightarrow$  Outputs `Relevant chunks: 3`.
- 10 chunks relevant  $\rightarrow$  `num_chunks = 10`  $\rightarrow$  Outputs `Relevant chunks: 10`.

## 6. Source Filename Problem Explained

### Old Behavior:

`rag_pipeline.py` checked `if "I could not find" in answer: sources = []`. If Ollama generated a refusal string, Python wiped the source list to `[]`, making `format_response()` output `Source: No relevant source found`, even though 10 chunks from `linux.pdf` had been retrieved!

### New Behavior:

Python extracts source filenames directly from `doc["path"]`:

```
sources = sorted(list(set(os.path.basename(doc["path"]) for doc in results)))
```

Python owns the source list. Ollama cannot erase it.

## 7. Future Uploaded Files

The pipeline uses **zero hardcoded filenames or chapter numbers**.

When a new file (e.g., `python_notes.pdf` or `company_data.xlsx`) is dropped into `watched_folder/`:

1. Celery processes the file and stores its chunks in LanceDB with `path = "/path/to/python_notes.pdf"`.
2. When a query mentions `"python_notes.pdf"`, `vector_search.py` dynamically matches `os.path.basename(path)` against the query string.
3. If no filename is mentioned, standard semantic vector search runs across all documents.
4. Source attribution extracts `os.path.basename(doc["path"])` dynamically ( `Source: python_notes.pdf` ).

## 8. Broad Query vs. Specific Query

- **Specific Query** ( *"What are user groups?"* ): Direct semantic match to body chunks.
- **Broad Meta-Query** ( *"Give me a summary of Chapter 1"* ):

\* Broad questions require information distributed across multiple body chunks.

\* Vector search does not need a single chunk containing the complete chapter summary.

*The updated prompt guidelines instruct Ollama: "You may synthesize an answer across multiple retrieved chunks."\**

## 9. The "No Relevant Source Found" Mismatch Explained

### The Old Bug Chain:

1. `vector_search.py` found 10 chunks  $\rightarrow$  printed `Found 10 relevant chunks` on terminal stdout.
2. `rag_pipeline.py` passed 10 chunks to Ollama.
3. Ollama issued a refusal: *"I could not find enough relevant information..."*.
4. `rag_pipeline.py` line 77 triggered `if "I could not find" in answer: sources = []`.
5. `format_response()` received `sources = []`  $\rightarrow$  printed `Source: No relevant source found`.

The Fix:

We removed the refusal override `if "I could not find" in answer: sources = []` and added a pre-LLM check `if num_chunks == 0:`. Python determines source metadata independently of Ollama's generated output.

10. Ollama's Role vs. Python's Role

| Responsibility                   | Python | Ollama |
|----------------------------------|--------|--------|
| Search database                  | YES    | NO     |
| Decide relevant chunks           | YES    | NO     |
| Count relevant chunks            | YES    | NO     |
| Determine source filename        | YES    | NO     |
| Build context string             | YES    | NO     |
| Generate natural language answer | NO     | YES    |
| Follow context-only rule         | NO     | YES    |

11. RAG Pipeline Data Flow Example

User Query: "What are the three main levels of a Linux system?"

1. User Query: "What are the three main levels of a Linux system?"

↓

2. Vector Embedding: MiniLM-L6-v2 converts query into 384-float vector

↓

3. LanceDB Search: `table.search(vector).limit(50)` fetches 50 candidate chunks

↓

4. Demotion & Filtering: `is_table_of_contents_or_noise()` demotes TOC; filters `_distance ≤ 0.50`

↓

5. Relevant Results: 10 body chunks selected

↓

6. Python Metadata Calculation: `num_chunks = 10, sources = ["linux.pdf"]`

↓

7. Pre-LLM Check: `num_chunks > 0` → Proceed to Ollama

↓

8. Context & Prompt Assembly: Formats 10 chunks into context string

↓

9. Ollama Execution: Generates natural language answer ("Hardware, Kernel, User Space")

↓

10. Python Formatting: `format_response()` appends Answer, Source: linux.pdf, Relevant chunks: 10

12. Actual Code Explanation

`search(question, max_results=10, score_threshold=0.50)`

- Purpose:** Performs semantic search in LanceDB and filters noise/thresholds.
- Input:** Query string `question`.
- Output:** List of relevant chunk dictionaries.
- Where called:** `rag_pipeline.py` line 38.

`ask(question)`

- **Purpose:** Coordinates query execution, vector search, prompt creation, and LLM call.
- **Input:** User query.
- **Output:** Tuple (`answer_string`, `list_of_sources`, `num_relevant_chunks`).
- **Where called:** Application UI / entry point.

```
format_response(answer, sources, num_chunks)
```

- **Purpose:** Formats response string for user display.
- **Input:** `answer`, `sources`, `num_chunks`.
- **Output:** Formatted markdown string.

### 13. Important Python Concepts Used

1. **List** (`[]`): Ordered collection of items (e.g. `relevant_chunks = []`).
2. **Dictionary** (`{}`): Key-value pairs (e.g. `{"page": 1, "text": "..."}` ).
3. **Set** (`set()`): Collection of unique elements used to remove duplicate filenames (`retrieved_sources = set()`).
4. **List Comprehension:** Compact loop filtering (e.g. `[c for c in raw_chunks if c["_distance"] ≤ 0.50]`).
5. `os.path.basename(path)`: Extracts file name from a full file system path (e.g. `/path/to/linux.pdf`  $\rightarrow$  `linux.pdf`).
6. `.get(key, default)`: Safely retrieves dictionary value without raising `KeyError` (e.g. `doc.get("_distance", 1.0)`).

### 14. Important RAG Concepts

- **Embedding:** Dense vector representation of text semantics.
- **Vector Distance (Cosine):** Measure of semantic dissimilarity (\$0.0\$ = identical, \$1.0\$ = unrelated).
- **Relevance Threshold:** Upper distance limit (\$0.50\$) for accepting chunks as relevant.
- **Context:** Text block built from retrieved chunks and passed to the LLM.
- **Hallucination:** LLM generating ungrounded facts not present in retrieved context.
- **Source Attribution:** Identifying the exact file from which context originated.

### 15. Before/After Real Examples

#### Scenario 1: Broad Summary Query

- **Query:** "Give me a summary of Chapter 1 of How Linux Works."
- **BEFORE:**
  - \* **Result:** Found 10 relevant chunks on stdout, but answer printed "I could not find enough relevant information", Source: No relevant source found.
  - \* **WHY:** TOC chunks crowded out body text, and Ollama's refusal wiped the source list in Python.
- **AFTER:**
  - \* **Result:** Synthesized answer covering User Space, Kernel, and Hardware.

#### Scenario 2: Irrelevant Question

- **Query:** "Who is Donald Trump?"
- **BEFORE:** Found 0 relevant chunks  $\rightarrow$  "I could not find..."
- **AFTER:**
  - \* **Answer:** Not found in the retrieved source.

### 16. Test Checklist for Current System

| Test | Question                           | Expected Retrieval | Expected Source          | Expected Behavior           | Bug Indicator             |
|------|------------------------------------|--------------------|--------------------------|-----------------------------|---------------------------|
| A    | "What are user groups in linux?"   | 10 chunks          | linux.pdf                | Answers grounded in text    | Refusal or missing source |
| B    | "Give me a summary of Chapter 1"   | 10 chunks          | linux.pdf                | Synthesizes chapter summary | Refusal string issued     |
| C    | "Who is Donald Trump?"             | 0 chunks           | No relevant source found | Reports Relevant chunks: 0  | Chunks retrieved > 0      |
| D    | "Which IT companies are in Excel?" | 10 chunks          | IT_Direct_Hire_...xlsx   | Lists companies             | PDF source listed         |
| E    | New uploaded .txt file query       | 1 chunk            | new_file.txt             | Answers from new file       | Old file attributed       |

17. Interview Question & Answer Bank

LEVEL 1 — Beginner

- Q: Why can't Ollama count retrieved chunks?  
A:\* Ollama is a text generator, not a database manager. Python performs the search and knows the exact length of the results list ( len(results) ). Asking LLMs to count leads to hallucinated numbers.
- Q: Why shouldn't source filenames come from Ollama?  
A:\* Ollama might misread or hallucinate filenames. Python tracks exact file paths from stored database metadata.

LEVEL 2 — Understanding

- Q: What happens when retrieval returns 0 chunks?  
A:\* Python detects num\_chunks == 0 before calling Ollama, returning Answer: Not found in the retrieved source. , Source: No relevant source found. , Relevant chunks: 0 .
- Q: Why do broad questions require multiple chunks?  
A:\* A book chapter spans multiple pages. No single 600-character chunk contains the entire chapter summary, so the LLM must synthesize information across multiple body chunks.

LEVEL 3 — Code & RAG Mechanics

- Q: What does a cosine distance score mean?  
A:\* Cosine distance measures dissimilarity between query and chunk vectors. Lower distance means higher semantic similarity.
- Q: What happens if score\_threshold is set too strict (e.g. 0.10)?  
A:\* Valid relevant chunks will be rejected, causing false refusals.
- Q: What happens if score\_threshold is set too loose (e.g. 0.90)?  
A:\* Irrelevant noise chunks will enter the context, degrading LLM accuracy.

18. Debugging Scenarios

- Scenario 1: Terminal stdout says Found 5 relevant chunks , but UI prints Source: No relevant source found .
  - Investigation: Check rag\_pipeline.py . Ensure there is no string check (like if "I could not find" in answer: sources = [] ) that clears sources after calling Ollama.
  - Correction: Ensure sources is calculated from Python search results and passed directly to format\_response() .
- Scenario 2: Broad chapter question returns Not found in the retrieved source .

- **Investigation:** Inspect `vector_search.py` candidate pool. Check if Table of Contents dotted lines crowded out body chunks.
- **Correction:** Expand candidate pool ( `limit(50)` ) and demote TOC/review noise.

---

## 19. Common Mistakes to Avoid

---

1. **Hardcoding Filenames:** Never write `if filename == "linux.pdf"`. Always use dynamic metadata ( `os.path.basename(doc["path"])` ).
2. **Forcing Fixed Chunk Counts:** Never pad results to force 10 chunks. Return only chunks meeting `_distance ≤ 0.50`.
3. **Letting LLM Control Metadata:** Never let Ollama report chunk counts or source filenames. Python owns retrieval metadata.
4. **Erasing Sources on Refusal:** Never wipe out `sources` if Ollama outputs a refusal string.

---

## 20. Final "Understand the Code" Summary

---

### Before (Old Architecture):

1. Fixed character slicing cut words in half.
2. PDF page numbers were discarded.
3. Raw `.limit(10)` vector search was crowded out by Table of Contents dotted lines.
4. String check `if "I could not find" in answer: sources = []` wiped out source filenames.
5. Strict prompt rules forced Ollama to refuse broad summary queries.

### After (New Architecture):

1. Paragraph-aware chunking ( `max_chars=600` ) keeps words and sentences intact.
3. Expanded candidate pool ( `limit(50)` ) demotes TOC noise, prioritizing body text chunks.
4. Python calculates `num_chunks` and `sources` before calling Ollama.
5. Ollama prompt explicitly allows multi-chunk synthesis for broad questions.
6. Unified output format ( `Answer` , `Source` , `Relevant chunks` ).

### What You Should Remember for Interviews:

- "Python owns retrieval truth; LLM owns natural language generation."
- "Cosine distance thresholding ( `0.50` ) ensures dynamic chunk counts (0 to 10)."
- "Generic metadata-driven design handles future files automatically without code edits."